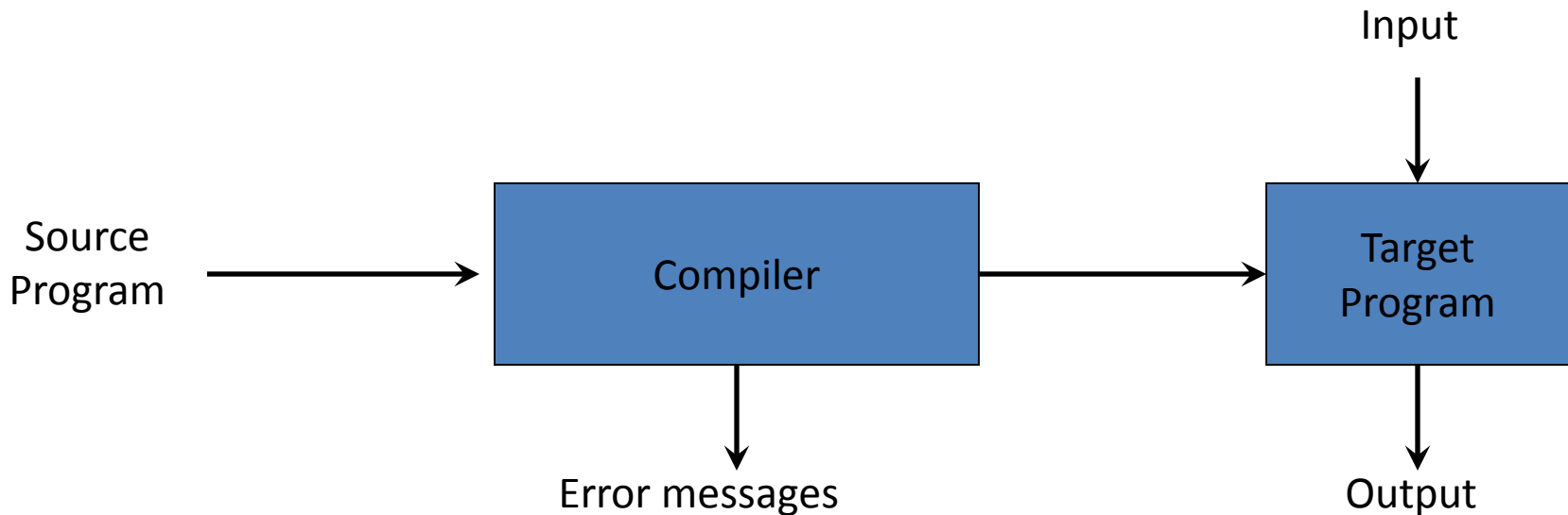


Know your compilers

# Compilers and Interpreters

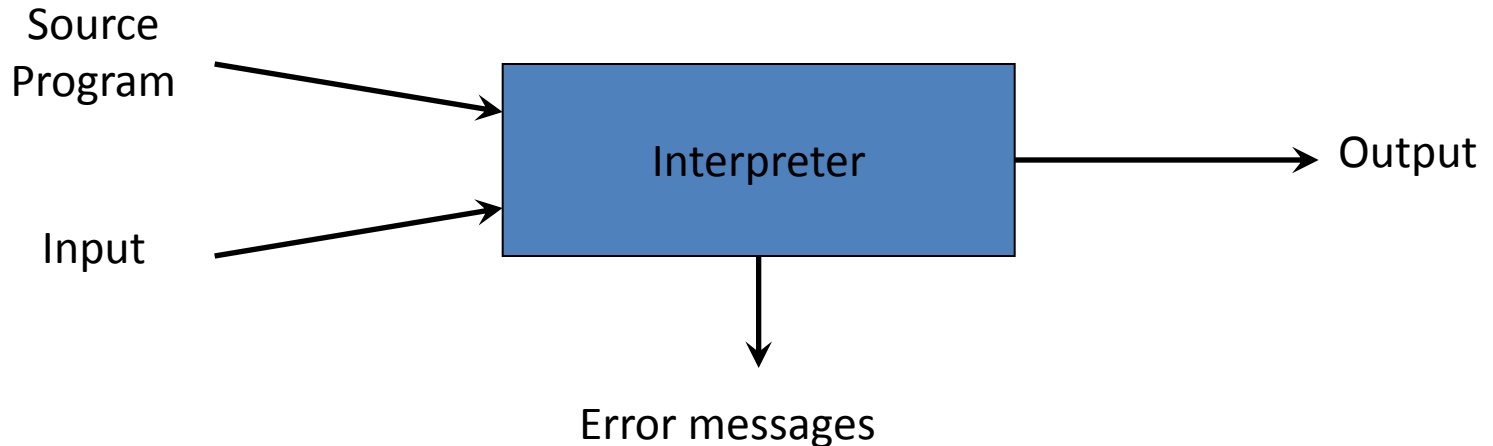
- **“Compiler”**

- Translation of a program written in a source language into a semantically equivalent program written in a target language



# Compilers and Interpreters

- **“Interpreter”**
  - Analyze and execute a program line by line.



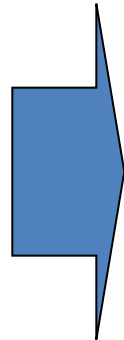
# Compilers and Interpreters

Source code → **Compiler** → Executable

Input data → **Executable** → Output data

Source code

Input data



**Interpreter**

→ Output data

## COMPILER

Fast, creates executable file that runs directly on the CPU

Debugging is more difficult. One error can produce many spurious errors

More likely to crash the computer. The machine code is running directly on the CPU

Easier to protect Intellectual Property as the machine code is difficult to understand

Uses more memory - all the execution code needs to be loaded into memory, although tricks like Dynamic Link Libraries lessen this problem

Unauthorised modification to the code more difficult. The executable is in the form of machine code. So it is difficult to understand program flow.

## INTERPRETER

Slower, interprets code one line at a time

Debugging is easier. Each line of code is analysed and checked before being executed

Less likely to crash as the instructions are being carried out either on the interpreters' command line or within a virtual machine environment which is protecting the computer from being directly accessed by the code.

Weaker Intellectual property as the source code (or bytecode) has to be available at run time. For example if you write a Flash Actionscript application, you can easily get de-compilers that convert the p-code back into actionscript source code (unless you use encryption, but that is another story).

Uses less memory, source code only has to be present one line at a time in memory

Easier to modify as the instructions are at a high level and so the program flow is easier to understand and modify.

# Compilers and Interpreters

- Most languages are usually thought of as using either one or the other:
  - Compilers: FORTRAN, COBOL, C, C++, Pascal
  - Interpreters: Lisp, scheme, BASIC, Perl, Python, Smalltalk
- BUT: not always implemented this way
  - Java and bytecode , Just-in-time compiling
  - Compilation may be followed by interpretation.
- Perl (Compiled to detect errors before interpretation)
- Java (Compiled to platform independent byte codes, and interpreted by the JVM)

# Compilers Vs. Interpreters

## ❑ *Compilation can be more efficient than interpretation*

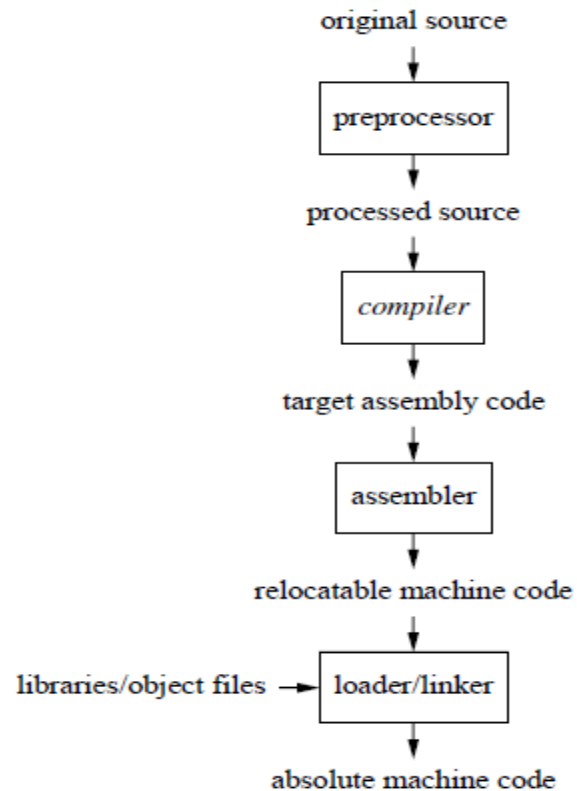
- ❖ Unlike a compiler, which translates the source program once and for all, an interpreter examines the program repeatedly.

## ❑ *Interpretation can be more flexible than compilation*

- ❖ An interpreter allows programs to be changed ``on the fly" to add features or correct errors.
- ❖ It can also pinpoint an error in the source text and report it accurately.

# A Language processing System

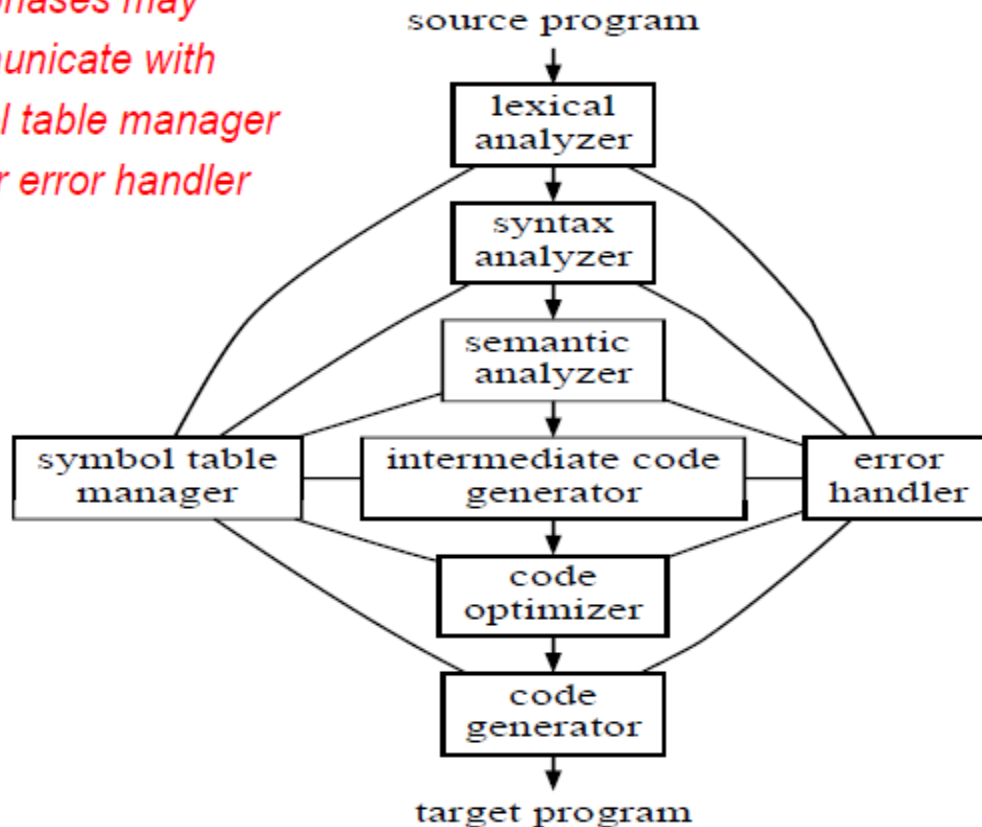
- Some tools often work in conjunction with compilers (e.g. assemblers, linkers, pre-processors)
- Often, they are coupled with compiler



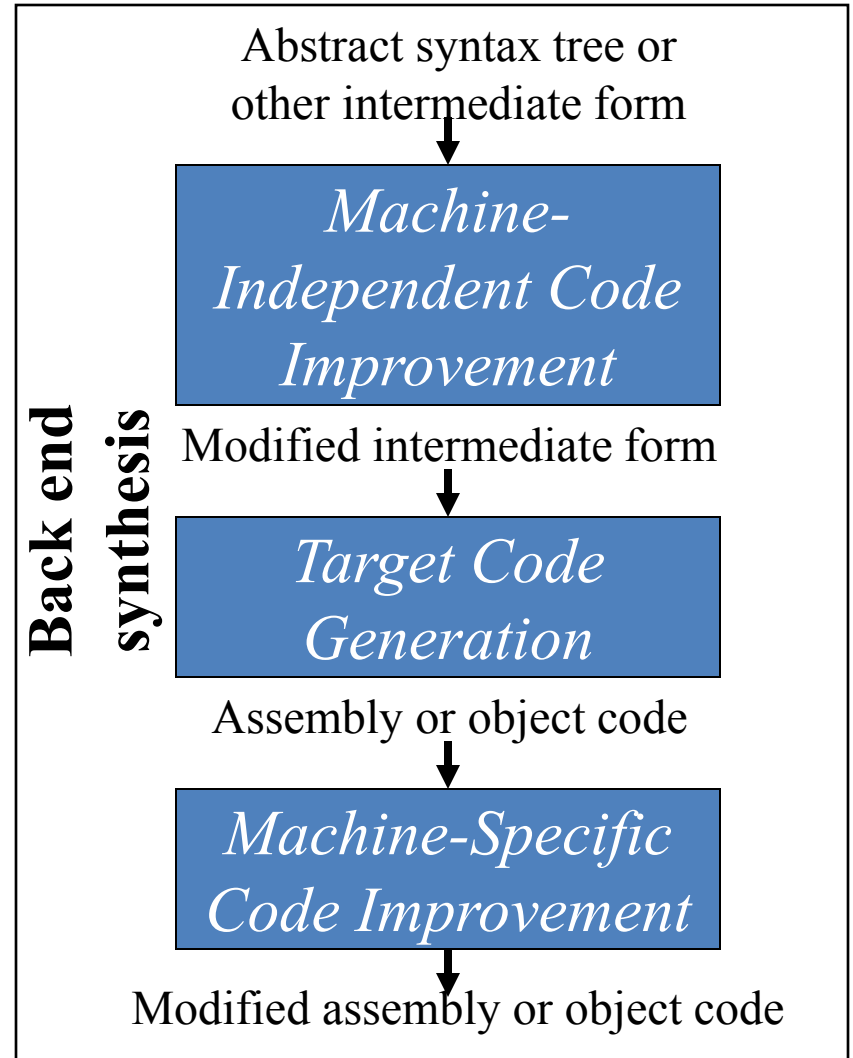
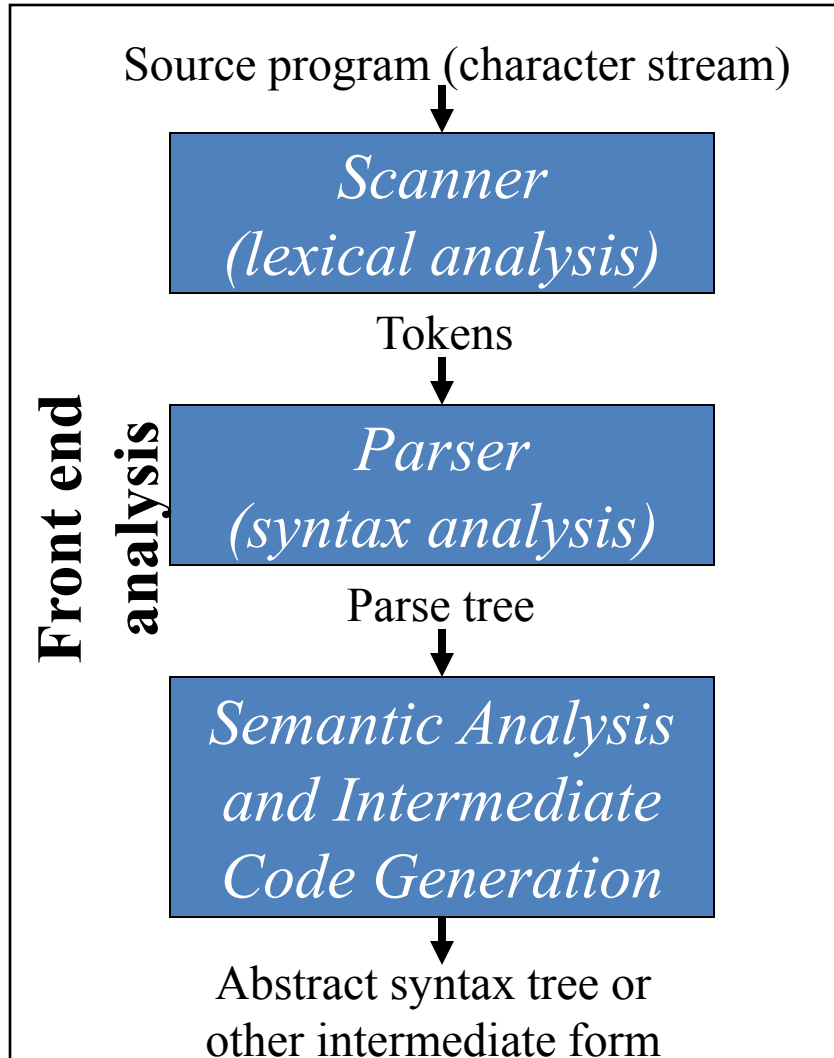


# Phases in compilers

- *All phases may communicate with symbol table manager and/or error handler*



# Compiler Front- and Back-end



# Front-end vs. back-end

- Front end deals with source language
  - Includes analysis, creation of symbol table, generation of intermediate code, some optimization
  - Independent of target machine
- Back end deals with target code
  - Includes some optimization, code generation
  - Depends on target machine, not on source language

# Lexical Analyzer (lexer)

- **Lexical Analyzer**
  - reads the source program character by character
  - returns the *tokens of the source program*
- The **lexemes** of a programming language include its identifiers, constants, operators, special symbols, keywords.
- A **token** of a language is a category of its lexemes.

# Lexical Analyzer

- Consider the following C statement: `new = 2 * old + 10;`

| <u>Lexemes</u> | <u>Tokens</u> |
|----------------|---------------|
| new            | identifier    |
| =              | equal-sign    |
| 2              | int-constant  |
| *              | mult-op       |
| old            | identifier    |
| +              | plus-op       |
| 10             | int-constant  |
| ;              | semicolon     |

# Lexical Analyzer

- Puts information about identifiers into the symbol table
- Tools:
  - Regular expressions used to describe tokens (lexical constructs)
  - (Deterministic) Finite State Automaton can be used in the implementation of a lexical analyzer

# Syntax analyzer (Parser)

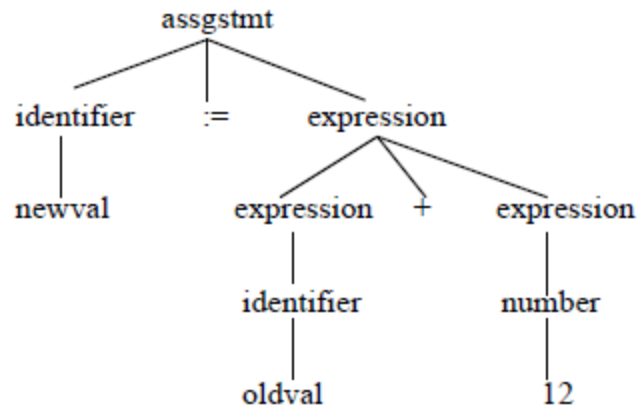
- Syntax of a language is specified by a **context free grammar (CFG)**
  - Rules in a CFG- mostly recursive
- A syntax analyzer checks whether a given program satisfies the rules implied by a CFG or not
  - creates the syntactic structure (generally a parse tree) of the given program

# Syntax analyzer

- `newval := oldval + 12`

We use BNF (Backus Naur Form) to specify a CFG

```
assgstmt -> identifier := expression
expression -> identifier
expression -> number
expression -> expression + expression
```



- In a parse tree, all terminals are at leaves
- All inner nodes are non-terminals in a context free grammar



# Syntax analyzer

- Different parsing techniques exist depending upon the way of how the parse tree is created
- Two main categories
  - *Top-Down Parsing*
  - *Bottom-Up Parsing*
- **Top-Down Parsing:**
  - Construction of the parse tree starts at the root, and proceeds towards the leaves
  - Efficient top-down parsers can be easily constructed by hand
  - E.g. Recursive Predictive Parsing, Non-Recursive Predictive Parsing (LL Parsing)
- **Bottom-Up Parsing:**
  - Construction of the parse tree starts at the leaves, and proceeds towards the root
  - Efficient bottom-up parsers are created with the help of some software tools
  - Bottom-up parsing is also known as shift-reduce parsing
  - Operator-Precedence Parsing – simple, restrictive, easy to implement
  - LR Parsing – much general form of shift-reduce parsing: LR, SLR, LALR

# Lexer Vs. Parser

- Both do similar things but at the *different levels*
- Lexical analyzer deals with simple non-recursive constructs of the language
- Syntax analyzer deals with recursive constructs of the language
- Lexical analyzer simplifies the job of the syntax analyzer
  - Lexical analyzer recognizes the smallest meaningful units (tokens) in a source program
  - Syntax analyzer works on the smallest meaningful units (tokens) in a source program to recognize meaningful structures in our programming language

# Semantic analysis

- Lexically and syntactically correct programs may still contain other errors
  - Are all variables, functions, classes, etc properly defined?
  - Are all expressions and variables used in ways that respect the type system?
  - Are the access control respected in the program?
- Checks source program for **semantic errors**

# Semantic Analysis

- Static semantic checks (done by the compiler) are performed at compile time
  - Type checking
  - Every variable is declared before used
  - Identifiers are used in appropriate contexts
  - Check subroutine call arguments
  - Check labels
- Dynamic semantic checks are performed at run time, and the compiler produces code that performs these checks
  - Array subscript values are within bounds
  - Arithmetic errors, e.g. division by zero
  - Pointers are not dereferenced unless pointing to valid object
  - A variable is used but hasn't been initialized
  - When a check fails at run time, an exception is raised

# Symbol-Table

- A mechanism (or, data structure) that allows information to be associated with identifiers and shared among compiler phases
  - Identifier declaration
  - Identifier use
  - Type checking
- Attributes for variables
  - storage, type, scope, etc.
- Attributes for procedures
  - name, parameters, etc.

*When a lexical analyzer sees an identifier for the first time, it adds it to the symbol table*

# Intermediate code generator

- Compiler may produce an explicit intermediate codes to represent source program.
- Intermediate codes are generally machine (architecture) independent.
- Should have at least two qualities:
  - Easy to produce
  - Easy to translate into target program

# Intermediate code generator

- **Example:** Three-address code

$i = 2*j + k - 1;$

↓

$t_1 = 2 * j$

$t_2 = t_1 + k$

$i = t_2 - 1$

- Implementation of three-address code:  
Quadruples

$t_1 = \text{minus } c$

$t_2 = b * t_1$

$t_3 = \text{minus } c$

$t_4 = b * t_3$

$t_5 = t_2 + t_4$

$a = t_5$

|   | <i>op</i> | <i>arg<sub>1</sub></i> | <i>arg<sub>2</sub></i> | <i>result</i>  |
|---|-----------|------------------------|------------------------|----------------|
| 0 | minus     | c                      |                        | t <sub>1</sub> |
| 1 | *         | b                      | t <sub>1</sub>         | t <sub>2</sub> |
| 2 | minus     | c                      |                        | t <sub>3</sub> |
| 3 | *         | b                      | t <sub>3</sub>         | t <sub>4</sub> |
| 4 | +         | t <sub>2</sub>         | t <sub>4</sub>         | t <sub>5</sub> |
| 5 | =         | t <sub>5</sub>         |                        | a              |
|   |           | ...                    |                        |                |

# Code optimizer

- Code optimization operates on intermediate code
  - **Improve intermediate code** by producing code that runs faster
  - Essential in terms of space and time



# Code optimizer

- **Analysis:** Compute information about a program
- **Semantics-preserving Transformations:** Use that information to perform program transformations  
(with the goal of improving some metric, e.g. performance)

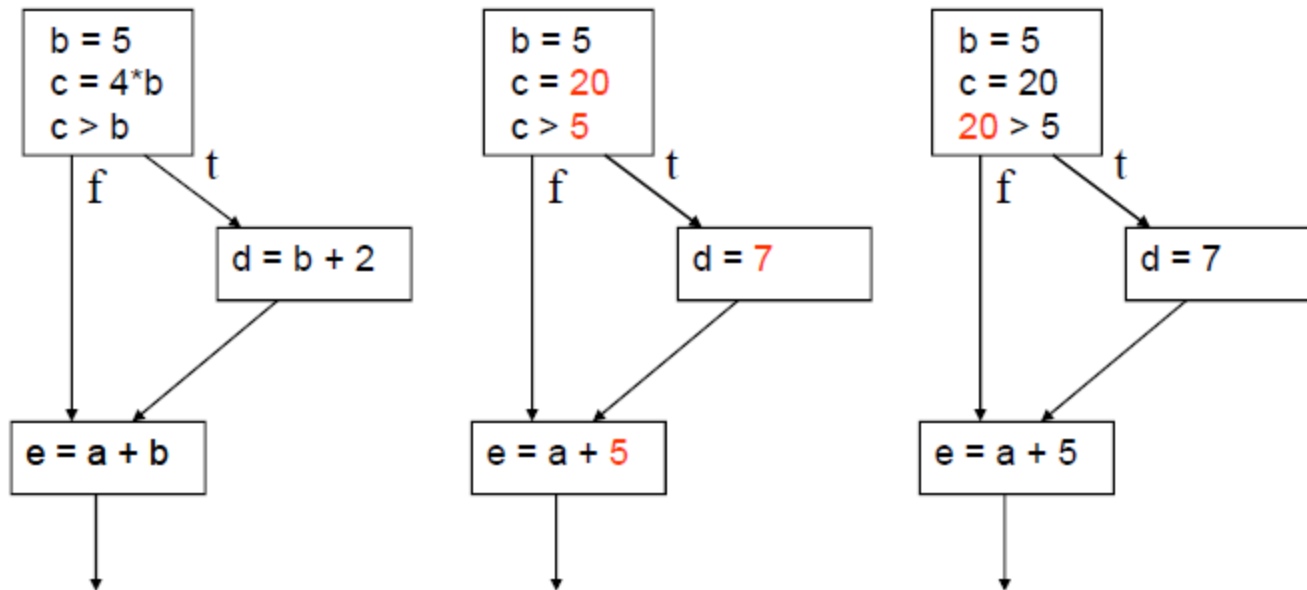
# Code optimizer : example

- Constant folding

|                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------|
| <pre>area := (22.0/7.0) * r ** 2</pre> <p style="text-align: center;">↓</p> <pre>area := 3.14286 * r ** 2</pre> |
|-----------------------------------------------------------------------------------------------------------------|

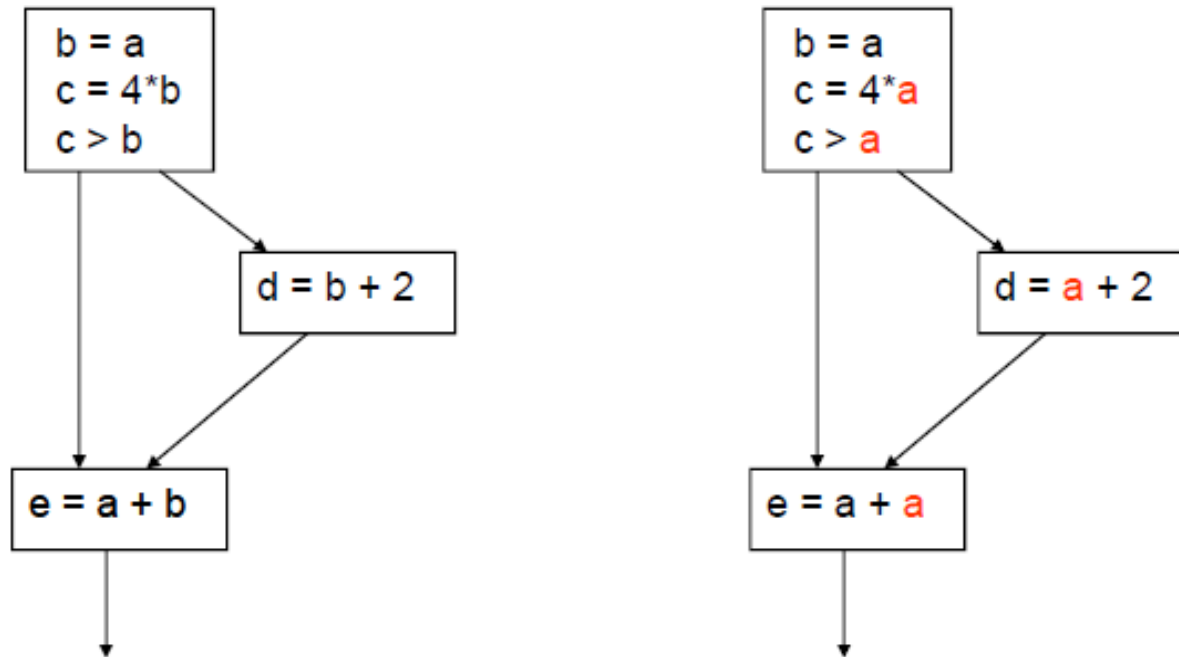
# Code optimizer : example

- Constant propagation



# Code optimizer : example

- Copy Propagation



# Code optimizer : example

- Common subexpression elimination

|                |   |               |
|----------------|---|---------------|
| a := b * c     |   | temp := b * c |
| ...            |   | a := temp     |
| ...            | → | ...           |
| x := b * c + 5 |   | x := temp + 5 |

# Code optimizer : example

- Dead code elimination

```
#define NNN 12  
if ( NNN > 10 )  
    x = 1.0;  
else  
    x = 2.0;
```



```
x = 1.0;
```

```
y = 5.0;  
x = 3.0;  
x = 4.0;  
printf("%f \n",x);
```



```
y = 5.0;  
x = 4.0;  
printf("%f \n",x);
```

# Code Generator

- Produces the target language in a specific architecture
- Target program is normally a relocatable object file
- Object file contain the machine codes

Ex: Assume that we have an architecture with instructions whose at least one of its operands is a machine register

```
MOVE    id2,R1
MULT     id3,R1
ADD      #1,R1
MOVE     R1,id1
```

# Machine-dependent optimization

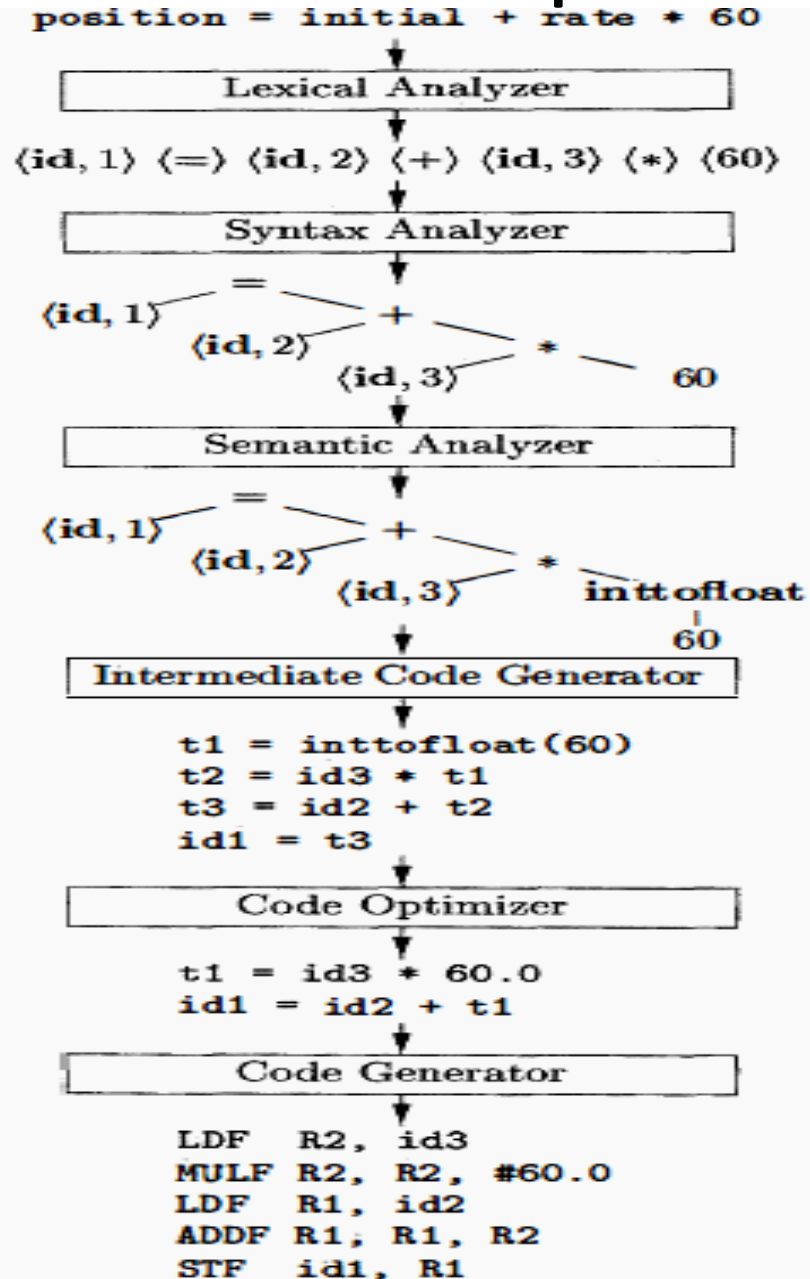
- Make improvements that require specific knowledge of machine architecture, e.g.
  - Optimize use of available registers
  - Reorder instructions to avoid waits



# Compiler Phases: example

|          |     |
|----------|-----|
| position | ... |
| initial  | ... |
| rate     | ... |
|          |     |

SYMBOL TABLE



# Multi-Pass Compilers

- Many compilers make multiple passes
  - Several phases often grouped into single pass (e.g. all of analysis and intermediate code generation)
  - Desirable to have relatively few passes
- Backpatching
  - Leaves blanks for unknown values to be filled in later
  - Allows merging of phases

# Compiler-Construction Tools

- Software development tools are available to implement one or more compiler phases
  - Scanner generators
  - Parser generators
  - Syntax-directed translation engines
  - Automatic code generators
  - Data-flow engines
- Example of two tools:
  - Lexical analyzers (e.g. Lex)
  - Compiler-compilers (e.g. Yacc)

# Programming Environments

- Collection of tools used in the development of software.
- Consists of a file system, a text editor, a linker, a compiler, a debugger, and development tools.
- Helps increase productivity and quality.
- Examples: Borland C++, Visual C++, Visual J++

# Programming environment tools

- editors (structure?)
  - vi, emacs ...
- pretty printers (formatting conventions)
  - cb, indent ...
- pre-processors
  - cpp, m4, watfor ...
- debuggers (needs symbol table)
  - adb, sdb, dbx, gdb ...
- style checkers
  - lint, purify, ...
- module management
  - make
- version management
  - sccs, rcs
- assemblers
  - as
- link editors, loaders
  - ld, ld-so

# Programming environment tools

- Visual Studio on PCs
- Cedar, Smalltalk, Interlisp at Xerox PARC
- Common Lisp on Symbolics
- Sabre C on Unix
- MPW on Macs